Software Performance Engineering



Recitation 2.1

Atharva Kirkole
Thursday, October 8, 2026

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Attendance!



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Some due dates

- Team contract of proj 2 due TODAY! do it here if not yet.
- One group only need to submit 1 copy(all names)
- Submit on GradeScope and include it in your GitHub repo.

- Homework 4 and Project 2 teams have been released,
- HW 4 due on 19th Oct
- Proj 2 Beta due 3rd Nov

Great job on Proj1, highest tier 46 on our end,

Commits matter. the final grade will depend on commit ratios, so try to keep it MINIMUM 1/3rd for 2 people teams and 15% for 3.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Team 1	Dang · Gottschall · Nguyen
Team 2	Biggar · Wheeler
Team 3	Pham · Verma
Team 4	Breissinger · Onakanmi
Team 5	Czopp · Nandamuri
Team 6	Menzies · Mulla
Team 7	Betten · Laidlaw
Team 8	Rexha · Vibber
Team 9	Kalani · Ranawat
Team 10	Moudgil · Thomas

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Project 2: Rendering and Simulating 3D Scenes



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Check Off

Clone your project repo and run:

```
make clean; make
./bin/ref-test tiers/1/s tiers/1/r
./bin/find-tier
```

The first is the correctness test, and the second is the performance test.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

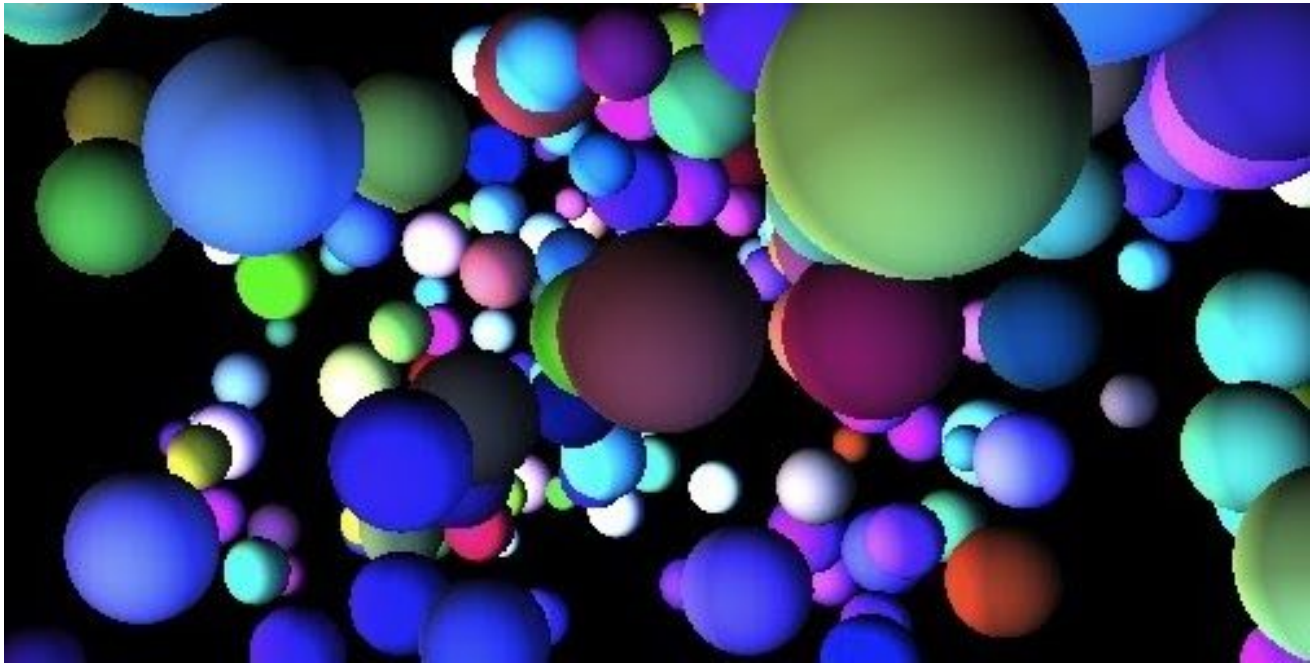
Example Output

```
ref-test options:
        s_spec = tiers/1/s
        r_spec = tiers/1/r
        renderer = student
        simulator = student
        n_frames = 12
Average difference: 0.0000000000000000
Standard deviation: 0.0000000000000000
Minimum difference: 0.0000000000000000
Maximum difference: 0.0000000000000000
Test result: PASS
```

```
PASS (woot!):   Tier 40 :   Ran 1536x1536   image with 3840 bodies in 388ms (s: 176, r: 211, is: 0, ir: 0)
PASS (ayy!):    Tier 41 :   Ran 1584x1584   image with 3994 bodies in 428ms (s: 190, r: 237, is: 0, ir: 0)
PASS (yeee!):   Tier 42 :   Ran 1648x1648   image with 4154 bodies in 431ms (s: 203, r: 225, is: 0, ir: 0)
PASS (yay!):    Tier 43 :   Ran 1728x1728   image with 4320 bodies in 497ms (s: 224, r: 269, is: 0, ir: 0)
PASS (eoo!):    Tier 44 :   Ran 1792x1792   image with 4493 bodies in 540ms (s: 240, r: 298, is: 0, ir: 0)
PASS (skrrt!):  Tier 45 :   Ran 1856x1856   image with 4672 bodies in 601ms (s: 261, r: 337, is: 0, ir: 0)
PASS (eoo!):    Tier 46 :   Ran 1936x1936   image with 4859 bodies in 598ms (s: 280, r: 315, is: 0, ir: 0)
PASS (boyah!):  Tier 47 :   Ran 2016x2016   image with 5054 bodies in 678ms (s: 304, r: 371, is: 0, ir: 0)
PASS (boyah!):  Tier 48 :   Ran 2096x2096   image with 5256 bodies in 722ms (s: 333, r: 386, is: 0, ir: 0)
```

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Project 2: N body simulation



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Idea

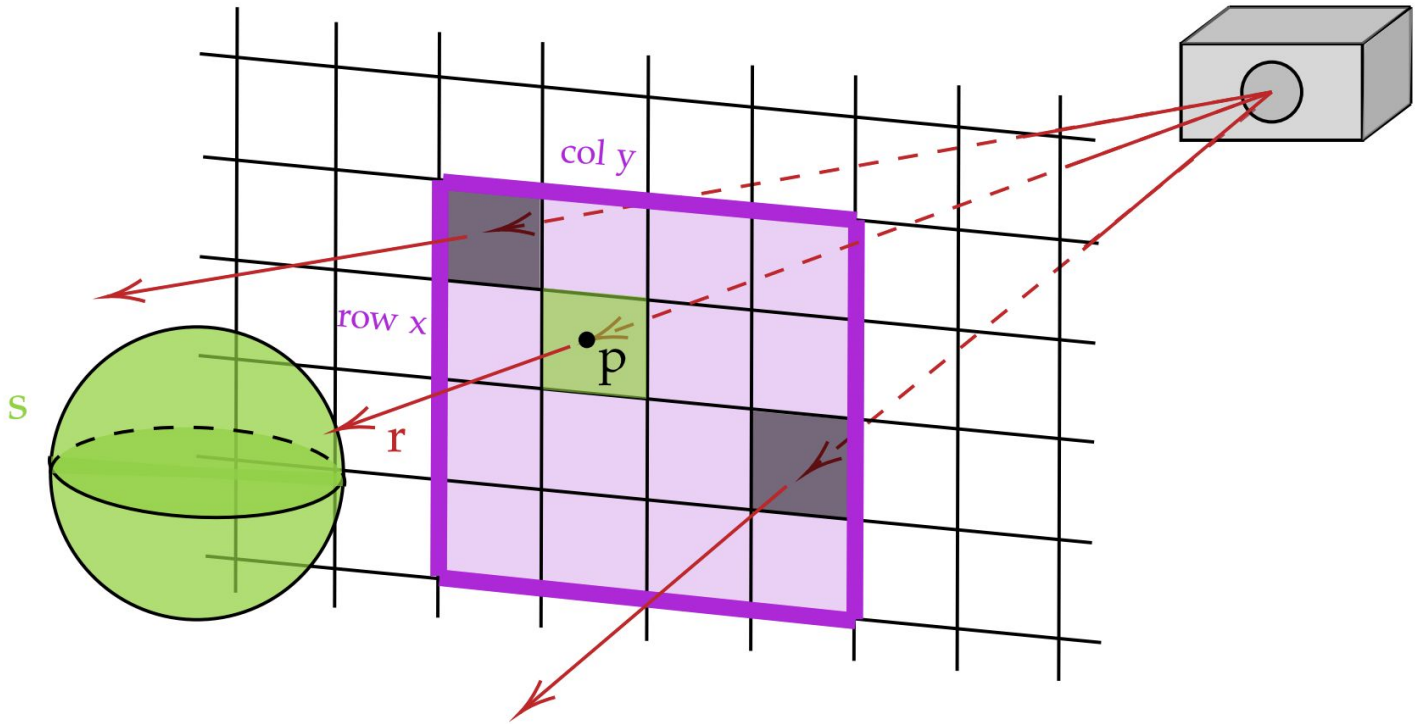
- **Drawing the picture (`render`)**: currently, for every single pixel on screen, it checks every single sphere: "am I looking at this one? this one? this one?" With 800 spheres and a 320×320 image that's 80 million checks per frame.

- The fix is to flip the question around. Instead of asking each *pixel* "what do I see?", ask each *sphere* "where do I land on screen?" then just color in that small patch. Each sphere only covers a little area, so you skip almost all the work.

- This is for rendering…. what about compute?

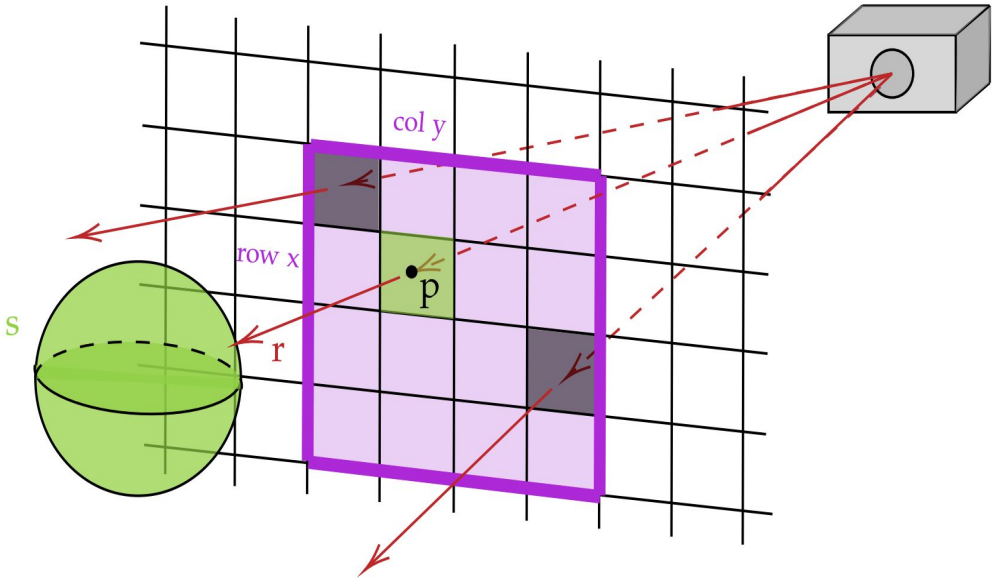
© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Ray Tracing



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

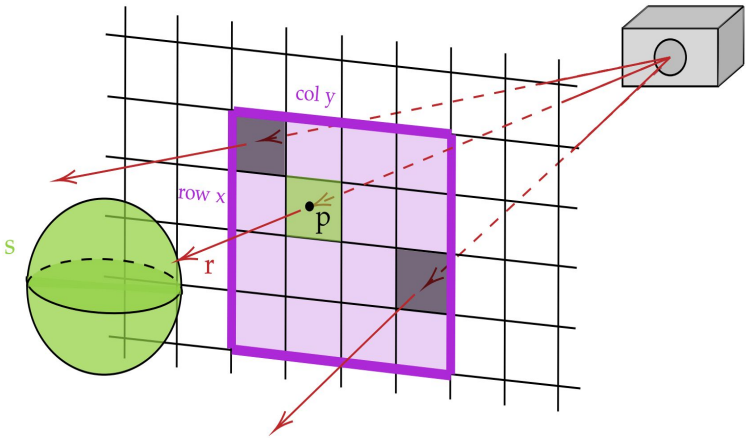
Elements of Ray Tracing



1. Origin (O)
2. Eye location (e)
3. Image Plane (can be inclined to the eye)
4. Plane orthonormal vectors (u and v)
5. Sphere : radius and center
6. Output Image is a resolution x resolution (square) image of pixels
7. Pixels have a fixed width, given by viewport/resolution

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

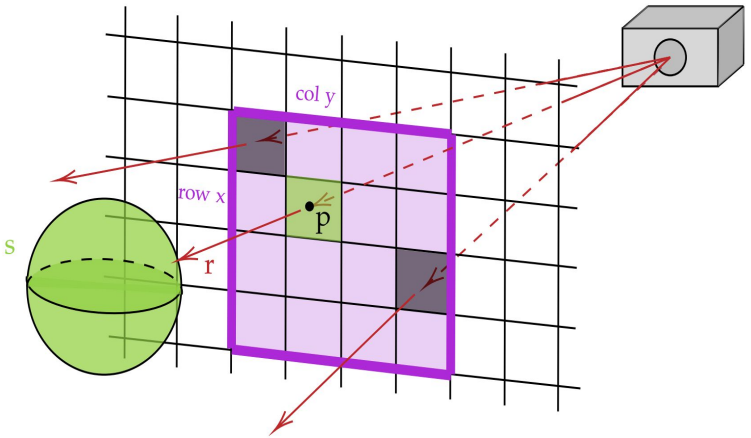
3 coordinates (2 spaces)



- X, Y, Z of the spheres
- X, Y, Z of the coordinates occurring on the plane (same units as spheres)
- I, J of the image (labeled x, y in the code)

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

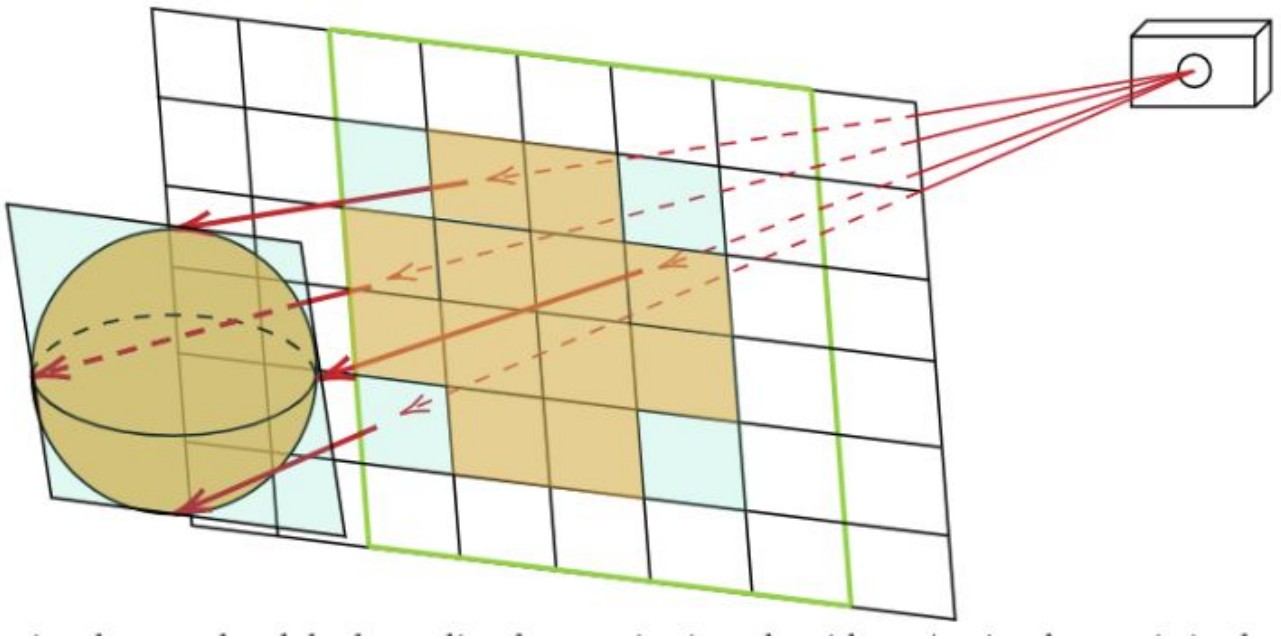
How do we get the pixel value at I and J?



- Given an image pixel location (I, J)
- Find the point in the 3D plane that corresponds to this pixel location (calculate ray from the eye)
- Determine if the ray hits a sphere
  - Complicated math

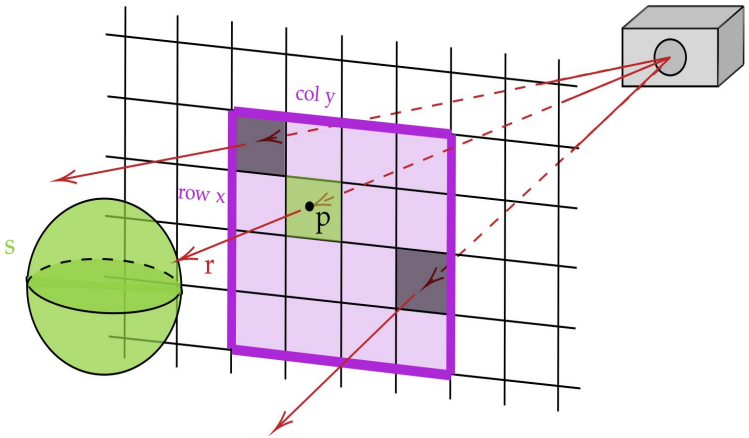
© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Bounding Boxes



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

A different way?



- Given a sphere location, get 8 points making a bounding cube
  - Ex: Given r and (x, y, z) some corners are (x+r, y+r, z+r), (x-r, y-r, z-r), (x+r, y-r, z-r)
- Project those points onto the plane
- For the bounding box of the cube in the image, check if the points intersect the sphere
  - If so, color the pixel

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Project the cube onto the plane

- Given points a, b find where they intersect a plane through the origin with orthonormal vectors u, v
- $\overleftrightarrow{ab} = \vec{b} + c * \left(\vec{a} - \vec{b}\right)$ for any number c
- Plane = $t\vec{u} + s\vec{v}$ for any numbers t, s
  - Also given by $p \cdot \vec{w} = 0$ where $\vec{w} = \vec{u} \times \vec{v}$
- 3 unknowns, 3 equations (one each for x, y, z components)
- Solve to get the 8 points and get a bounding box of those 8 points

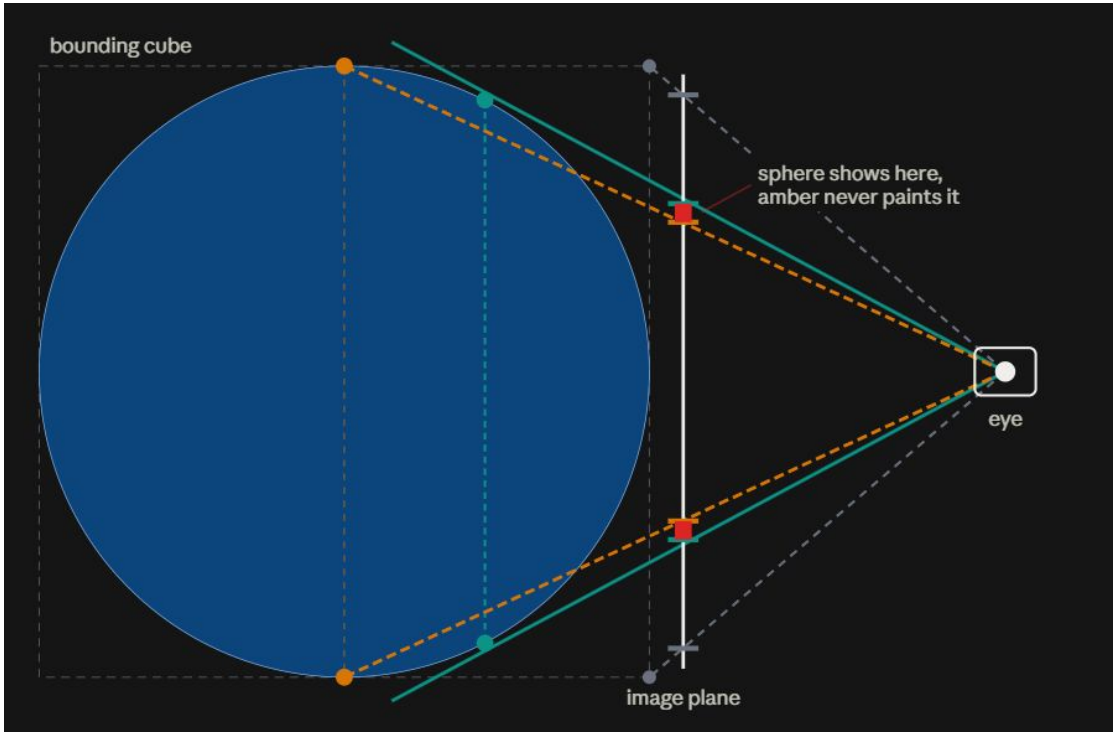
© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

CODING TIME!



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

why cube



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers